

a intra_inter_bonds

November 20, 2025

1 Inter- and intrachain bond counting

To see if there are bonding sites that are being used up by neighbouring, same-chain bonding sites, we will look at snapshots of trajectory files and compare the counts of interchain and intrachain bonding.

It's a bit tricky to figure out which beads belong to one chain, there is no ID attached to chains at the moment. Instead, knowing the number of beads in a single chain (N) and assuming they are generated and IDs are attached sequentially, we can assume that each chunk of N beads will belong to the same chain.

```
[4]: import numpy as np
from tqdm import tqdm

# number of beads in a chain = chain length +
amine_spacing = 3
chain_length = 30
N = chain_length + len(range(amine_spacing, chain_length, amine_spacing))

# read snapshot
def snapshot_generator(traj_file, start_ts=None, end_ts=None):
    """Yields (timestep, data) tuples from a LAMMPS trajectory file without
    loading it all."""
    with open(traj_file) as f:
        while True:
            line = f.readline()
            if not line:
                break
            if line.startswith("ITEM: TIMESTEP"):
                ts = int(f.readline().strip())
                if start_ts is not None and ts < start_ts:
                    continue
                if end_ts is not None and ts > end_ts:
                    break
            # Skip next header lines
            while not (l := f.readline()).startswith("ITEM: NUMBER OF_
            ATOMS"):
                continue
```

```

        n_atoms = int(f.readline().strip())
        # Skip to atom data
        while not (l := f.readline()).startswith("ITEM: ATOMS"):
            continue
        # Read atom data block
        data = np.loadtxt([f.readline() for _ in range(n_atoms)])
        yield ts, data

# check intrachain ids
def count_intra_inter_bonds(traj_file, start_ts=None, end_ts=None,
    ↪bond_cutoff=1.5):
    intra_count = 0
    inter_count = 0
    distances = []
    for ts, data in snapshot_generator(traj_file, start_ts, end_ts):
        print(f"Processing timestep {ts}")
        positions = data[:, 2:5] # assuming columns 2-4 are x,y,z
        ids = data[:, 0] # assuming column 0 is atom id
        chain_ids = ((ids - 1) // N).astype(int) # assuming N is the chain
    ↪length

        # not the fastest but we aren't processing a lot of snapshots
        n_atoms = len(positions)
        for i in tqdm(range(n_atoms)):
            for j in range(i + 1, n_atoms):
                dist = np.linalg.norm(positions[i] - positions[j])
                if dist < bond_cutoff:
                    if chain_ids[i] == chain_ids[j]:
                        intra_count += 1
                    else:
                        inter_count += 1
                distances.append(dist)
        print(f"Timestep {ts}: Intra-chain bonds: {intra_count}, Inter-chain
    ↪bonds: {inter_count}")
    return intra_count, inter_count, distances

```

Let's test this function out on a run with a bonding site density of 1/2 (every second bead has a reversible bonding site). Note that the number of bonds is counted up, so the ratio is a more effective comparison between different function calls.

```

[ ]: intra_count, inter_count, distances = count_intra_inter_bonds('logs/
    ↪patch_locations.dat', start_ts=12000000, bond_cutoff=0.2)

print(f"Total intra-chain bonds: {intra_count}")
print(f"Total inter-chain bonds: {inter_count}")

```

```
print(f"Ratio intra/inter: {intra_count / inter_count if inter_count > 0 else_
↳ 'Inf'}")
```

The ratio is 2.2, which means a significant amount (about a third) of the reversibly bonded pairs are between different chains. This supports the stiffer behaviour of the bonding density of $1/2$. Let us now compare it with $1/3$, which has some issues in the $C(t)$ data.

1.1 Bonding density $1/3$

There is something weird happening with spacing out bonding sites by two, such that one in three beads has a bonding site on it. The density values surrounding it ($1/4$, $1/2$) both display plateauing behaviour of $C(t)$, but $1/3$ appears decay as quickly as much lower density values.

```
[ ]: intra_count, inter_count, distances = count_intra_inter_bonds('logs/
↳ patch_locations.dat', start_ts=11900000, bond_cutoff=0.2)

print(f"Total intra-chain bonds: {intra_count}")
print(f"Total inter-chain bonds: {inter_count}")
print(f"Ratio intra/inter: {intra_count / inter_count if inter_count > 0 else_
↳ 'Inf'}")
```

The ratio of intra- to inter-bonds is 17.10098717803245, which is quite bad. This means only a small portion the bonded pairs of interaction sites are between chains, and are therefore restricting some of the stiff behaviour we would want to see.

Similarly, let's compare this to a bonding site density of $1/4$, which also displayed plateauing behaviours. In total, both $1/2$ and $1/4$ had some kind of plateauing, but $1/3$ did not in the plots of $C(t)$ I retrieved.

```
[ ]: intra_count, inter_count, distances = count_intra_inter_bonds('logs/
↳ patch_locations.dat', start_ts=11900000, bond_cutoff=0.2)

print(f"Total intra-chain bonds: {intra_count}")
print(f"Total inter-chain bonds: {inter_count}")
print(f"Ratio intra/inter: {intra_count / inter_count if inter_count > 0 else_
↳ 'Inf'}")
```

Comparing the ratios, it does look like the number of intra-chain bonds peaks at the density of $1/3$. This is strange, and might be described by the geometry of the beads. When there are two beads in between each bonding site, there can be bends that allow for adjacent bonding sites to bond, which is statistically more prone to happen than when they are more separated. When there is only a single site, it is more restricted for adjacent sites to be close enough to bond.

Bonding Site Density	Non-bonding beads \ per interval	num intrachain bonds	num interchain bonds	inter:intra ratio
1/2	1	236396	107597	2.19705
1/3	2	150711	8813	17.1010
1/4	3	6433	7508	0.85682